**PRÁCTICA DE JAVASCRIPT**

**Funciones, funciones flecha, métodos de arrays y manejo de errores**

**Introducción**

En esta práctica se trabajan conceptos fundamentales de JavaScript relacionados con las funciones y la manipulación de datos:

- Declaración y llamada de funciones.
- Funciones anónimas y funciones flecha.
- Funciones de orden superior.
- Métodos map() y filter().
- Comprobación de tipos con Array.isArray().
- Parámetros predeterminados.
- Operadores rest (...) y spread (...).
- Gestión de excepciones mediante throw, try, catch y finally.

Cada ejercicio incluye el código resuelto, una explicación de su funcionamiento, ejemplos de uso y los errores más habituales.

---

**EJERCICIO 1. VALIDADOR DE CONTRASEÑA**

**1.1. Enunciado**

Completar la función esContrasenaValida, que debe devolver true si la contraseña tiene ocho caracteres o más y false en caso contrario.

**1.2. Código resuelto**

```
function esContrasenaValida(contrasena) {
  return contrasena.length >= 8;
}

const contrasenas = ['1234', 'miClave2024', 'abc'];

const resultado = contrasenas.map(
  contrasena => esContrasenaValida(contrasena)
```

);

console.log(resultado);

// [false, true, false]

### 1.3. Explicación del código

**Declaración de la función**

function esContrasenaValida(contrasena) {

- function es la palabra reservada para declarar una función.
- esContrasenaValida es el nombre de la función.
- contrasena es el parámetro que recibe el valor que se desea comprobar.

**Comprobación de la longitud**

return contrasena.length >= 8;

- .length devuelve el número de caracteres de una cadena.
- >= significa «mayor o igual que».
- return devuelve el resultado de la comparación.

La comparación ya produce un valor booleano (true o false), por lo que no hace falta utilizar un if.

**Uso de map()**

const resultado = contrasenas.map(

contrasena => esContrasenaValida(contrasena)

);

El método map() recorre el array y crea uno nuevo aplicando una función a cada elemento.

La expresión:

contrasena => esContrasenaValida(contrasena)

es una función flecha anónima que recibe cada contraseña y llama a la función validadora.

El resultado es un array de valores booleanos.

| Contraseña | Longitud | Resultado |
|---|---|---|
| '1234' | 4 | false |
| 'miClave2024' | 11 | true |
| 'abc' | 3 | false |

**1.4. Funciones con nombre y funciones anónimas**

Una función con nombre permite reutilizar una operación y facilita su identificación en el código.

```
function duplicar(numero) {
  return numero * 2;
}


console.log(duplicar(5)); // 10
```

Una función anónima puede utilizarse cuando la operación se necesita en un contexto concreto, por ejemplo, como argumento de map().

```
const numeros = [1, 2, 3];


const dobles = numeros.map(
  numero => numero * 2
);


console.log(dobles); // [2, 4, 6]
```

También se puede pasar directamente la función con nombre:

```
const resultado = contrasenas.map(esContrasenaValida);
```

Esta última opción es más sencilla porque map() entrega cada elemento como argumento a la función.

**¿Cuándo conviene utilizar cada una?**

- Funciones con nombre: cuando la operación se reutiliza, tiene cierta complejidad o merece una identificación clara.

- Funciones anónimas: cuando se necesita una operación breve y específica, especialmente como callback de otro método.

**1.5. Errores frecuentes**

- Utilizar > en lugar de >=: una contraseña de exactamente ocho caracteres no se aceptaría.
- Olvidar return: la función no devolvería el resultado esperado.
- Confundir .length con el índice de un elemento.
- Escribir map() sin devolver el resultado de la función callback.

**Caso de uso:** comprobar requisitos mínimos de longitud en formularios de registro. Esta validación no sustituye a otras medidas de seguridad, como comprobar contraseñas comprometidas o aplicar políticas adicionales.

---

**EJERCICIO 2. CALCULADORA POR FUNCIÓN**

**2.1. Enunciado**

Implementar las operaciones suma, resta y potencia mediante funciones flecha. Crear también aplicarOperacion, una función de orden superior que reciba dos números y una función que indique qué operación realizar.

La función potencia debe lanzar un error si el exponente es negativo.

**2.2. Código resuelto**

```
const suma = (a, b) => a + b;


const resta = (a, b) => a - b;


const potencia = (base, exponente) => {
  if (exponente < 0) {
    throw new Error(
      'El exponente no puede ser negativo'
    );
  }
```

```
  return base ** exponente;
};

const aplicarOperacion = (a, b, operacion) =>
  operacion(a, b);

console.log(aplicarOperacion(5, 3, suma));     // 8
console.log(aplicarOperacion(5, 3, resta));    // 2
console.log(aplicarOperacion(2, 3, potencia)); // 8

console.log(aplicarOperacion(2, -1, potencia));
// Lanza un error: El exponente no puede ser negativo
```

La última llamada lanza una excepción. Si no se captura mediante try/catch, la ejecución normal del programa se interrumpe en ese punto.

**2.3. Explicación de las funciones flecha**

Las funciones flecha se escriben utilizando el operador =>.

Ejemplo:

```
const suma = (a, b) => a + b;
```

Equivale, en este caso, a:

```
function suma(a, b) {
  return a + b;
}
```

Cuando el cuerpo contiene una sola expresión, el resultado se devuelve automáticamente. No es necesario escribir return ni utilizar llaves.

Sin embargo, si se emplea un cuerpo de bloque con llaves, hay que escribir return cuando se desea devolver un resultado.

```
const resta = (a, b) => {
  return a - b;
};
```

**2.4. El operador de potencia**

return base ** exponente;

El operador ** calcula la potencia de un número.

Por ejemplo:

console.log(2 ** 3); // 8

console.log(5 ** 2); // 25

Antes de calcularla, se comprueba el exponente:

```
if (exponente < 0) {
  throw new Error(
    'El exponente no puede ser negativo'
  );
}
```

- if permite ejecutar un bloque si se cumple una condición.
- throw lanza una excepción.
- new Error(...) crea un objeto de error con un mensaje descriptivo.

**2.5. ¿Qué es una función de orden superior?**

Una función de orden superior es una función que recibe otra función como argumento o devuelve una función.

En este ejercicio:

```
const aplicarOperacion = (a, b, operacion) =>
  operacion(a, b);
```

El parámetro operacion contiene una función. Por eso podemos pasarle suma, resta o potencia.

Por ejemplo:

aplicarOperacion(5, 3, suma);

La función ejecuta internamente:

suma(5, 3);

Es importante pasar suma sin paréntesis. Si se escribiera suma(5, 3), se estaría pasando el resultado de la suma, no la función.

### 2.6. Errores frecuentes

- Olvidar el return dentro de una función flecha con cuerpo de bloque.
- Escribir operacion sin llamarla: debe ser operacion(a, b).
- Pasar suma(5, 3) en vez de suma.
- Olvidar el throw y permitir exponentes negativos.
- Pensar que throw muestra un mensaje y continúa normalmente: en realidad, lanza una excepción que debe gestionarse.

**Caso de uso:** una calculadora extensible, en la que se puede añadir una nueva operación sin modificar la estructura de aplicarOperacion.

---

## EJERCICIO 3. CATÁLOGO DE PRODUCTOS

### 3.1. Enunciado

A partir de un array de productos, se deben obtener:

1. Un array con los nombres de los productos que tienen stock disponible.
2. Una cadena de texto con una lista HTML que contenga esos nombres.

No se pueden utilizar bucles for.

### 3.2. Código resuelto

```
const productos = [
 { nombre: 'Ratón', precio: 15, stock: 0 },
 { nombre: 'Teclado', precio: 25, stock: 8 },
 { nombre: 'Monitor', precio: 120, stock: 3 }
];


const disponibles = productos
  .filter(producto => producto.stock > 0)
  .map(producto => producto.nombre);


const listaHtml = '<ul>' + disponibles
  .map(nombre => `<li>${nombre}</li>`)
```

```
  .join('') + '</ul>';


console.log(disponibles);

// ['Teclado', 'Monitor']


console.log(listaHtml);

// <ul><li>Teclado</li><li>Monitor</li></ul>
```

**3.3. El método filter()**

El método filter() crea un nuevo array con los elementos que cumplen una condición.

```
.filter(producto => producto.stock > 0)
```

La expresión comprueba que el stock sea mayor que cero.

- Ratón: stock igual a 0; se descarta.
- Teclado: stock igual a 8; se conserva.
- Monitor: stock igual a 3; se conserva.

El array original no se modifica.

**3.4. El método map()**

Después de filtrar, se selecciona únicamente la propiedad nombre.

```
.map(producto => producto.nombre);
```

El array pasa de contener objetos completos a contener cadenas de texto.

Antes:

```
[
  { nombre: 'Teclado', precio: 25, stock: 8 },
  { nombre: 'Monitor', precio: 120, stock: 3 }
]
```

Después:

```
['Teclado', 'Monitor']
```

**3.5. Encadenamiento de métodos**

Es posible aplicar varios métodos consecutivamente porque filter() devuelve un array y ese array permite llamar a map().

```
const disponibles = productos
 .filter(producto => producto.stock > 0)
 .map(producto => producto.nombre);
```

El proceso es:

1. Obtener los productos con stock.
2. Extraer sus nombres.
3. Guardar el nuevo array en disponibles.

Este estilo hace que el código sea más declarativo: expresa qué resultado se busca en lugar de describir paso a paso cómo recorrer los datos.

**3.6. Construcción de la lista HTML**

```
const listaHtml = '<ul>' + disponibles
 .map(nombre => `<li>${nombre}</li>`)
 .join('') + '</ul>';
```

La cadena se construye mediante tres operaciones.

**Primero: transformar cada nombre en un elemento HTML.**

```
.map(nombre => `<li>${nombre}</li>`)
```

Las comillas invertidas permiten utilizar plantillas de texto. La expresión ${nombre} se sustituye por el valor de la variable.

**Segundo: unir los elementos.**

```
.join('')
```

El método join() convierte los elementos del array en una sola cadena. El argumento vacío indica que no debe añadirse ningún separador.

**Tercero: añadir el contenedor.**

```
'<ul>' + ... + '</ul>'
```

Se colocan las etiquetas de apertura y cierre de la lista.

**3.7. Errores frecuentes**

- Utilizar stock >= 0, lo que incluiría productos sin existencias.

- Intentar acceder a producto.nombre después de haber convertido los objetos en cadenas.
- Olvidar join(''), obteniendo un array en vez de una cadena.
- Confundir una cadena que contiene HTML con elementos HTML insertados en una página.

**Caso de uso:** generar listados de productos, inventarios, catálogos o resultados de búsqueda.

**Nota de seguridad:** si los nombres proceden de usuarios o de fuentes no confiables, deben escaparse correctamente antes de insertarlos como HTML en una página web. Construir una cadena no equivale a insertarla de forma segura en el DOM.

---

**EJERCICIO 4. SUMA FLEXIBLE**

**4.1. Enunciado**

Implementar sumaFlexible(x, y) para aceptar números o arrays de números.

Si un argumento es un array, debe sumarse su contenido. Si es un número, debe conservarse su valor.

Se utilizará Array.isArray() para comprobar el tipo real del argumento.

**4.2. Código resuelto**

```
'use strict';


const sumaFlexible = (x, y) => {
 const valorDe = (v) => {
  if (Array.isArray(v)) {
   return v.reduce(
    (acumulado, numero) => acumulado + numero,
    0
   );
  }
```

```
  return v;
 };


 return valorDe(x) + valorDe(y);
};


console.log(sumaFlexible(3, 4));     // 7
console.log(sumaFlexible([1, 2], 4)); // 7
console.log(sumaFlexible([1, 2], [3, 4])); // 10
```

**4.3. Comprobación con Array.isArray()**

```
Array.isArray(v)
```

Devuelve true si v es un array y false en caso contrario.

Ejemplos:

```
console.log(Array.isArray([1, 2])); // true
console.log(Array.isArray(5));     // false
```

La función auxiliar valorDe() permite trabajar de la misma manera con ambos tipos de argumentos.

Si recibe un array, lo suma. Si recibe un número, lo devuelve sin modificarlo.

**4.4. El método reduce()**

```
v.reduce(
 (acumulado, numero) => acumulado + numero,
 0
);
```

reduce() recorre el array y acumula sus valores en un único resultado.

Por ejemplo, para [1, 2, 3]:

| Paso | Acumulado anterior | Número | Nuevo acumulado |
|---|---|---|---|
| 1 | 0 | 1 | 1 |

| Paso | Acumulado anterior | Número | Nuevo acumulado |
|---|---|---|---|
| 2 | 1 | 2 | 3 |
| 3 | 3 | 3 | 6 |

El 0 final es el valor inicial del acumulador. Por ello, un array vacío produce una suma de 0.

**4.5. ¿Por qué se utiliza una función auxiliar?**

const valorDe = (v) => { … };

Esta función evita repetir la comprobación de tipo para x e y.

Finalmente:

return valorDe(x) + valorDe(y);

Se obtiene un valor numérico de cada argumento y se suman ambos resultados.

**4.6. Errores frecuentes**

- Utilizar typeof v === 'array': no funciona porque typeof devuelve "object" para los arrays.
- Olvidar el valor inicial 0 en reduce().
- No devolver v cuando no es un array.
- Dar por supuesto que los argumentos siempre contienen números.

**Importante:** la solución presupone que los argumentos son números o arrays cuyos elementos son números. Para una función de producción habría que validar también los argumentos y sus elementos.

**Caso de uso:** sumar cantidades procedentes de diferentes fuentes, donde algunas se reciben individualmente y otras agrupadas en arrays.

---

**EJERCICIO 5. CREACIÓN DE USUARIOS**

**5.1. Enunciado**

Completar crearUsuario() para que reciba un nombre obligatorio y un rol opcional. Si no se indica el rol, se asignará automáticamente el valor 'alumno'.

**5.2. Código resuelto**

const crearUsuario = (nombre, rol = 'alumno') => ({

```
  nombre,
  rol
});

console.log(crearUsuario('Ana'));
// { nombre: 'Ana', rol: 'alumno' }

console.log(crearUsuario('Luis', 'admin'));
// { nombre: 'Luis', rol: 'admin' }
```

**5.3. Parámetros predeterminados**

La expresión:

```
rol = 'alumno'
```

establece un valor predeterminado para el parámetro rol.

Cuando se llama a la función sin proporcionar ese argumento, se utiliza el valor definido.

```
crearUsuario('Ana');
```

Es equivalente a:

```
crearUsuario('Ana', 'alumno');
```

Si se proporciona otro rol, se utilizará el valor recibido.

```
crearUsuario('Luis', 'admin');
```

**5.4. Retorno implícito de un objeto**

```
const crearUsuario = (nombre, rol = 'alumno') => ({
  nombre,
  rol
});
```

Las llaves que contienen el objeto están entre paréntesis porque, en una función flecha, las llaves directamente después de => se interpretan como el inicio de un bloque de código.

Los nombres abreviados:

{ nombre, rol }

son una forma abreviada de escribir:

```
{
  nombre: nombre,
  rol: rol
}
```

Ambas formas crean el mismo objeto.

**5.5. Errores frecuentes**

- Olvidar las comillas de 'alumno'.
- Escribir el objeto sin paréntesis, provocando que las llaves se interpreten como un bloque.
- Confundir un parámetro predeterminado con una comprobación obligatoria.
- Pensar que el valor predeterminado se aplica cuando se pasa cualquier valor falso: se aplica cuando el argumento se omite o recibe undefined, pero no cuando se pasa null.

**Caso de uso:** crear usuarios, configurar opciones predeterminadas o inicializar objetos de una aplicación.

**Nota:** en una aplicación real, el nombre y el rol deberían validarse. Un valor predeterminado no impide que alguien proporcione un rol no permitido.

---

**EJERCICIO 6. MÁXIMO SIN Math.max()**

**6.1. Enunciado**

Crear una función maximo() que utilice el parámetro rest para recibir una cantidad variable de números. La función debe encontrar el mayor valor mediante un bucle.

Después, se debe llamar a la función utilizando el operador spread sobre un array existente.

**6.2. Código resuelto**

```
const maximo = (...numeros) => {
  if (numeros.length === 0) {
    throw new Error(
```

```
    'Debe proporcionarse al menos un número'
  );
 }

 let mayor = numeros[0];

 for (let i = 1; i < numeros.length; i++) {
  if (numeros[i] > mayor) {
   mayor = numeros[i];
  }
 }

 return mayor;
};

const notas = [7, 9, 5, 10, 6];

console.log(maximo(...notas)); // 10
```

**6.3. Parámetro rest**

```
const maximo = (...numeros) => {
```

El parámetro rest permite recibir un número variable de argumentos y agruparlos en un array.

Por ejemplo:

```
console.log(maximo(7, 9, 5, 10, 6)); // 10
```

Dentro de la función, numeros contiene:

```
[7, 9, 5, 10, 6]
```

**6.4. Búsqueda del valor máximo**

Primero se inicializa la variable mayor con el primer número:

```
let mayor = numeros[0];
```

Después se recorren los elementos restantes:

```
for (let i = 1; i < numeros.length; i++) {
  if (numeros[i] > mayor) {
    mayor = numeros[i];
  }
}
```

En cada iteración se compara el número actual con el mayor encontrado hasta ese momento. Si el número actual es superior, se actualiza la variable.

Al terminar el recorrido, mayor contiene el valor máximo.

Se comprueba también que el array no esté vacío. Si no se proporciona ningún argumento, se lanza un error en vez de devolver un resultado engañoso.

**6.5. Operador spread**

La llamada:

```
maximo(...notas);
```

utiliza el operador spread para expandir los elementos del array y pasarlos como argumentos individuales.

Es equivalente a escribir:

```
maximo(7, 9, 5, 10, 6);
```

**6.6. Diferencias entre rest y spread**

Aunque utilizan la misma sintaxis (...), tienen funciones distintas.

**Rest: agrupa argumentos.**

```
const sumarTodos = (...numeros) => {
  return numeros.reduce(
    (total, numero) => total + numero,
    0
  );
};
```

```
console.log(sumarTodos(1, 2, 3)); // 6
```

Los argumentos individuales se agrupan en un array.

**Spread: expande elementos.**

```
const numeros = [1, 2, 3];


console.log(sumarTodos(...numeros)); // 6
```

Los elementos del array se pasan como argumentos individuales.

| **Característica** | **Rest** | **Spread** |
|---|---|---|
| Función | Agrupar | Expandir |
| Uso habitual | Definición de funciones | Llamadas a funciones |
| Resultado | Un array de argumentos | Argumentos o elementos separados |

### 6.7. Errores frecuentes

- Confundir ...numeros en la definición con ...notas en la llamada.
- Inicializar mayor en 0, lo que daría un resultado incorrecto si todos los números fueran negativos.
- Olvidar devolver el máximo.
- No contemplar el caso de una llamada sin argumentos.
- Utilizar el índice 0 para empezar el bucle y volver a comparar innecesariamente el primer elemento.

**Caso de uso:** calcular la nota más alta de un grupo, el precio máximo de una lista o el valor más alto de una serie numérica.

---

## EJERCICIO 7. DIVISIÓN SEGURA

### 7.1. Enunciado

Implementar una función dividir(a, b) que lance una excepción si el divisor es cero.

Después, capturar la excepción utilizando try/catch y ejecutar un bloque finally que indique que la operación ha finalizado.

### 7.2. Código resuelto

```
const dividir = (a, b) => {
```

```
  if (b === 0) {
    throw new Error(
      'No se puede dividir entre cero'
    );
  }

  return a / b;
};

try {
  console.log(dividir(10, 0));
} catch (e) {
  console.log('Error:', e.message);
} finally {
  console.log('Operación finalizada');
}
```

Salida:

Error: No se puede dividir entre cero

Operación finalizada

La división no llega a ejecutarse porque la función lanza una excepción cuando detecta que el divisor es cero.

**7.3. La comprobación con if**

```
if (b === 0) {
  throw new Error(
    'No se puede dividir entre cero'
  );
}
```

El operador === compara tanto el valor como el tipo, sin realizar conversiones automáticas.

Si el divisor es exactamente el número 0, se lanza el error.

Si la condición no se cumple, se ejecuta:

return a / b;

**7.4. Gestión de errores con try/catch/finally**

**Bloque try**

```
try {
  console.log(dividir(10, 0));
}
```

Contiene el código que puede provocar una excepción.

**Bloque catch**

```
catch (e) {
  console.log('Error:', e.message);
}
```

Captura la excepción lanzada por throw.

La variable e contiene el objeto de error. Su propiedad message permite acceder al mensaje descriptivo.

Por ejemplo:

```
const error = new Error('Ha ocurrido un problema');


console.log(error.message);
// Ha ocurrido un problema
```

**Bloque finally**

```
finally {
  console.log('Operación finalizada');
}
```

Se ejecuta después del bloque try y, cuando corresponde, después de catch, tanto si la operación termina correctamente como si se produce una excepción.

Es útil para ejecutar tareas de limpieza o indicar que un proceso ha finalizado.

**7.5. Ejemplo de división correcta**

```
try {
  console.log(dividir(10, 2));
} catch (e) {
  console.log('Error:', e.message);
} finally {
  console.log('Operación finalizada');
}
```

Salida:

5

Operación finalizada

En este caso no se ejecuta catch, porque no se produce ninguna excepción. El bloque finally sí se ejecuta.

**7.6. Errores frecuentes**

- Escribir b = 0 en lugar de b === 0: la primera expresión asigna un valor, no comprueba la igualdad.
- Lanzar una excepción sin capturarla cuando se desea controlar el error.
- Escribir e en lugar de e.message cuando solo se quiere mostrar el mensaje.
- Pensar que finally se ejecuta únicamente cuando hay un error.
- Olvidar que throw interrumpe el flujo normal de la función hasta que la excepción se gestiona.

**Caso de uso:** validar operaciones matemáticas, gestionar fallos de entrada o controlar errores en procesos que necesitan ejecutar tareas finales de limpieza.

---

**CONCLUSIÓN GENERAL**

Los siete ejercicios permiten comprender cómo las funciones y los métodos de JavaScript ayudan a escribir código reutilizable, legible y fácil de mantener.

Los conceptos más importantes son:

1. **Funciones con nombre:** permiten reutilizar operaciones y mejorar la claridad del código.
2. **Funciones flecha:** ofrecen una sintaxis concisa para definir funciones.

3. **Funciones de orden superior:** permiten pasar funciones como argumentos y crear operaciones flexibles.
4. **map() y filter():** permiten transformar y seleccionar elementos de arrays sin utilizar bucles explícitos.
5. **Array.isArray() y reduce():** ayudan a distinguir arrays y a transformar una colección en un único valor.
6. **Parámetros predeterminados:** permiten definir valores cuando se omiten argumentos.
7. **Rest y spread:** permiten agrupar argumentos y expandir colecciones.
8. **Excepciones:** throw, try, catch y finally permiten detectar y gestionar situaciones excepcionales.

La elección de cada recurso depende del problema que se quiera resolver. Un buen programa no solo debe producir el resultado correcto, sino también ser comprensible, controlar las situaciones inesperadas y facilitar futuras modificaciones.

**Recomendaciones finales**

- Utilizar nombres de variables y funciones descriptivos.
- Mantener una indentación uniforme.
- Escribir comentarios para explicar decisiones importantes, no para repetir literalmente cada línea.
- Validar los datos recibidos cuando la función vaya a utilizarse en una aplicación real.
- Probar tanto los casos habituales como los casos límite, por ejemplo, arrays vacíos, valores negativos o argumentos no válidos.
- Separar la lógica de negocio de la presentación de los resultados cuando el programa crezca.

**Fin de la práctica.**